

Farg'ona viloyati, O'zbekiston tumani, Qumbosti MFY
«KELAJAK TEXNOLOGIYALARI»
Nazorat ishlari — To'liq Javoblar va Baholash Mezonlari
8-sinf | 4 chorak | O'qituvchi uchun qo'llanma

Tuzuvchi:	Xo'jayev Muhammad Yusuf
Tekshiruvchi:	Maslahatchi D. Mehmonova
O'quv yili:	2026–2027
Hujjat turi:	O'qituvchi nusxasi (javoblar bilan)

Diqqat: Yashil rangdagi javoblar — to'liq namuna. O'quvchi javobi mazmunan mos kelsa, to'liq ball beriladi. So'zma-so'z mos bo'lishi shart emas.

I CHORAK NAZORAT ISHI — Oktabr oxiri

Format: Yozma test (5 savol) + Amaliy topshiriq (2 topshiriq) | Jami: 20 ball

NAZARIY QISM — 10 ball

1. NumPy massivi (ndarray) Python ro'yxatidan nimasi bilan farq qiladi?

Python ro'yxati — har xil turdagi elementlar, sekin, ko'p xotira.

NumPy ndarray — bir turdagi elementlar, tez (C da yozilgan), vektorlashtirilgan operatsiyalar.

```
import numpy as np
a = np.array([1, 2, 3, 4, 5])
print(a * 2)      # [2 4 6 8 10] – element bo'yicha ko'paytirish
print(a.mean())   # o'rtacha
print(a.std())     # standart og'ish
```

Kalit: NumPy = tezkor, matematik operatsiyalar uchun.

2. Pandas DataFrame'da ma'lumot tozalash: bo'sh qiymatlar (NaN) bilan qanday ishlash mumkin?

```
import pandas as pd
df = pd.read_csv('data.csv')
df.isnull().sum()      # qaysi ustunda nechta NaN bor
df.dropna()            # NaN bo'lgan qatorlarni o'chirish
df.fillna(0)           # NaN ni 0 bilan to'ldirish
df['yosh'].fillna(df['yosh'].mean()) # o'rtacha bilan to'ldirish
```

Kalit: NaN bo'lsa — o'chir yoki to'ldir. To'ldirish ko'pincha ma'qulroq.

3. Pandas'da groupby() qanday ishlaydi? Misolda tushuntiring.

groupby() — ma'lumotlarni guruhlab, har guruh bo'yicha hisob-kitob qiladi.

Misol — davlat bo'yicha o'rtacha maosh:

```
df.groupby('davlat')['maosh'].mean()
# yoki ko'p statistik:
df.groupby('sinf').agg({'baho': ['mean', 'max', 'min']})
```

Kalit: groupby = SQL'dagi GROUP BY ga o'xshash.

4. Matplotlib'da subplot nima? Bir oynada bir nechta grafik qanday chiziladi?

```
import matplotlib.pyplot as plt
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
ax1.plot([1,2,3], [4,5,6])
ax1.set_title('Chiziqli grafik')
ax2.bar(['A','B','C'], [3,7,2])
ax2.set_title('Ustunli grafik')
plt.tight_layout()
plt.show()
```

Kalit: subplots(qatorlar, ustunlar) — bir oynada ko'p grafik.

5. Seaborn'da heatmap qanday ishlaydi va qachon ishlatiladi?

Heatmap — ikki o'zgaruvchi orasidagi bog'liqlikni rang orqali ko'rsatadi.

```
import seaborn as sns
```

```
korrel = df.corr() # korrelyatsiya matritsasi
sns.heatmap(korrel, annot=True, cmap='coolwarm')
plt.show()
annot=True — har katakda raqamni ko'rsatadi.
```

Qachon: ko'p o'zgaruvchi orasidagi munosabatni ko'rish uchun.

Kalit: *Heatmap = korrelyatsiya matritasini vizuallashtirish.*

AMALIY QISM — 10 ball

1. Pandas bilan CSV faylni yuklab, ustunlar bo'yicha asosiy statistik ko'rsatkichlarni (o'rtacha, min, max, standart og'ish) chiqaring va 2 ta grafik chizing.

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('data.csv')
print(df.describe()) # barcha statistik ko'rsatkichlar
# Grafik 1:
df['narx'].hist(bins=20)
plt.title('Narx taqsimoti')
plt.show()
# Grafik 2:
df.groupby('kategoriya')['narx'].mean().plot(kind='bar')
plt.title('Kategoriya bo'yicha o'rtacha narx')
plt.show()
```

Kalit: *Baholash: describe() — 2, histogram — 1.5, bar chart — 1.5 ball*

2. NumPy bilan 5x5 tasodifiy matritsa yaratib, har qator va ustunning yig'indisini hisoblang.

```
import numpy as np

matritsa = np.random.randint(1, 100, size=(5, 5))
print("Matritsa:\n", matritsa)
print("Qatorlar yig'indisi:", matritsa.sum(axis=1))
print("Ustunlar yig'indisi:", matritsa.sum(axis=0))
print("Umumiy yig'indi:", matritsa.sum())
```

Kalit: *Baholash: random matritsa — 2, axis=1 va axis=0 — 3 ball*

II CHORAK NAZORAT ISHI — Dekabr oxiri

Format: Yozma test (5 savol) + Amaliy topshiriq | Jami: 20 ball

NAZARIY QISM — 10 ball

1. Mashina o'qishida 'nazorat ostidagi' va 'nazorat ostisiz' o'rganish farqi nima?

Nazorat ostida (Supervised) — o'quv ma'lumotlari yorliqlangan (label). Model kirish-chiqishni o'rganadi.

Misol: uy rasmiga 'mushuk' yorlig'i qo'yilgan.

Nazorat ostisiz (Unsupervised) — yorliqlar yo'q. Model o'zi guruhlarini topadi.

Misol: mijozlarni xarid odatiga ko'ra guruhlash (clustering).

Kalit: *Supervised = yorliqli; Unsupervised = guruhlarini o'zi topadi.*

2. Scikit-learn'da train_test_split nima uchun kerak? 80/20 nisbati nimani anglatadi?

train_test_split — ma'lumotlarni o'qitish (train) va tekshirish (test) qismlariga bo'ladi.

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

80/20 — 80% o'qitish, 20% tekshirish. Test qismi model hech ko'rmagan yangi ma'lumot rolini o'ynaydi.

Kalit: *Train — o'qitish; Test — mustaqil baholash.*

3. Chiziqli regressiya (Linear Regression) qanday ishlaydi?

Chiziqli regressiya — ikki o'zgaruvchi orasidagi to'g'ri chiziqli munosabatini modellaydi: $y = mx + b$

Misol: uy maydoni (X) → narxi (y)

```
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

```
predictions = model.predict(X_test)
```

Kalit: *LR — raqamli natija bashorati (narx, harorat, og'irlik).*

4. Klassifikatsiya modelida accuracy, precision va recall ko'rsatkichlari nima?

Accuracy — to'g'ri bashoratlar ulushi (jami ichida).

Precision — 'musbat' deb topilganlar orasida haqiqiy musbat ulushi (noto'g'ri signal kam).

Recall — barcha haqiqiy musbatlardan qanchasi topildi (o'tkazib yuborish kam).

```
from sklearn.metrics import classification_report
```

```
print(classification_report(y_test, y_pred))
```

Kalit: *Accuracy yetarli emas! Precision va Recall ham muhim.*

5. Decision Tree (Qaror daraxti) algoritmi qanday ishlaydi?

Decision Tree — ma'lumotni savollarga asosida bo'lib boradi (if-else daraxt).

Har qadam eng foydali bo'luvchi xususiyatni tanlaydi (Gini impurity yoki Information Gain).

```
from sklearn.tree import DecisionTreeClassifier
```

```
model = DecisionTreeClassifier(max_depth=5)
```

```
model.fit(X_train, y_train)
```

max_depth — daraxtning chuqurligini cheklaydi (overfitting oldini olish).

Kalit: *Qaror daraxti — vizual va tushunarli, lekin chuqurlik katta bo'lsa overfitting.*

AMALIY QISM — 10 ball**1. Scikit-learn bilan Iris dataset'dan KNN klassifikator o'qiting va accuracy ni hisoblang.**

```
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print('Accuracy:', accuracy_score(y_test, pred))
```

Kalit: Baholash: split — 1, model yaratish — 2, fit/predict/score — 2 ball

2. Uy narxini bashorat qiluvchi chiziqli regressiya modeli yarating (sodda dataset bilan) va natijani grafik orqali ko'rsating.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

maydon = np.array([[40],[60],[80],[100],[120]])
narx = np.array([50000, 75000, 100000, 130000, 155000])
model = LinearRegression()
model.fit(maydon, narx)
pred = model.predict([[90]])
print(f'90m2 narx: {pred[0]:.0f} so\'m')

plt.scatter(maydon, narx)
plt.plot(maydon, model.predict(maydon), 'r-')
plt.show()
```

Kalit: Baholash: model.fit() — 2, predict — 1, scatter+plot — 2 ball

III CHORAK NAZORAT ISHI — Mart oxiri

Format: Yozma test (5 savol) + Amaliy topshiriq | Jami: 20 ball

NAZARIY QISM — 10 ball

1. Neyron tarmoqda activation function nima uchun kerak?

Activation function — neyronning chiqishini nolinear qiladi va tarmoqqa murakkab naqshlarni o'rganish imkonini beradi.

Eng ko'p ishlatiladiganlar:

ReLU: $\max(0, x)$ — salbiyni nolga aylantiradi, tezkor

Sigmoid: $1/(1+e^{-x})$ — 0-1 oraliq (ikkilik klassifikatsiya)

Softmax — ko'p sinf klassifikatsiya (ehtimollar)

Linear activation bo'lmasa — ko'p qatlamli tarmoq ham bir qatlamga teng bo'ladi.

Kalit: *Activation = nolinearlik = murakkab naqshlarni o'rganish.*

2. Keras'da oddiy neyron tarmoq yaratishning ketma-ketligini yozing.

```
from tensorflow import keras
model = keras.Sequential([
    keras.layers.Dense(64, activation='relu', input_shape=(10,)),
    keras.layers.Dense(32, activation='relu'),
    keras.layers.Dense(1, activation='sigmoid')
])
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train, epochs=10, batch_size=32)
```

Kalit: *Sequential → Dense qatlamlar → compile → fit.*

3. NLP'da tokenization nima va nima uchun kerak?

Tokenization — matnni kichik bo'laklarga (token — so'z, harf, subso'z) bo'lish.

Misol: 'Men maktabga boraman' → ['Men', 'maktabga', 'boraman']

AI matn ustida to'g'ridan-to'g'ri ishlamaydi — avval raqamlarga aylantirish kerak. Tokenization birinchi qadam.

Zamonaviy modellarda BPE (Byte Pair Encoding) yoki WordPiece tokenizatsiya ishlatiladi.

Kalit: *Tokenization = matnni raqamlarga o'tkazishning birinchi qadami.*

4. Overfitting nima va uni oldini olish uchun qanday usullar bor?

Overfitting — model o'quv ma'lumotini yod olib qoladi, lekin yangi ma'lumotda yomon ishlaydi.

Belgisi: train accuracy yuqori, test accuracy past.

Oldini olish usullari:

1. Dropout qatlami (tasodifiy neyronlarni o'chirish)
2. Regularization (L1/L2)
3. Ko'proq o'quv ma'lumoti
4. Cross-validation
5. Early stopping

Kalit: *Overfitting = yodlash. Test accuracy — haqiqiy baholash.*

5. Generativ AI (ChatGPT, Stable Diffusion)ning asosidagi texnologiyalar haqida nima bilasiz?

ChatGPT asosida — Transformer arxitekturasi (GPT = Generative Pre-trained Transformer). Self-attention mexanizmi orqali so'zlar o'rtasidagi bog'liqlikni o'rganadi.

Stable Diffusion — diffuzion model: shovqindan bosqichma-bosqich rasm yaratadi.

Ikkalasi ham milliardlab parametrlı neyron tarmoqlar.

Kalit: *Transformer = zamonaviy AI asosi. Attention = muhim qismga e'tibor.*

AMALIY QISM — 10 ball

1. Keras bilan sodda ikkilik klassifikator yarating (masalan, 2 sinf o'rtasida) va training history'ni grafik orqali ko'rsating.

```
from tensorflow import keras
import numpy as np, matplotlib.pyplot as plt
X = np.random.randn(1000, 5)
y = (X[:, 0] + X[:, 1] > 0).astype(int)
model = keras.Sequential([
    keras.layers.Dense(16, activation='relu', input_shape=(5,)),
    keras.layers.Dense(1, activation='sigmoid')
])
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
history = model.fit(X, y, epochs=20, validation_split=0.2, verbose=0)
plt.plot(history.history['accuracy'], label='Train')
plt.plot(history.history['val_accuracy'], label='Val')
plt.legend(); plt.show()
```

Kalit: *Baholash: model yaratish — 2, compile/fit — 2, grafik — 1 ball*

2. Guruh bilan: ochiq NLP/ML dataset'dan (Kaggle yoki UCI) ma'lumot tahlili va oddiy model qurish. Natijani taqdim eting.

Baholash rubrikasi: dataset tanlash va tushuntirish — 2, EDA (grafik) — 3, model qurish va accuracy — 3, taqdimot — 2 ball.

Kalit: *Haqiqiy dataset bilan ishlash tajribasi baholanadi.*

IV CHORAK — YAKUNIY NAZORAT — May oxiri

Format: Yakuniy loyiha himoyasi + Yillik test | Jami: 30 ball

NAZARIY QISM — 10 ball

1. Docker nima va nima uchun dasturlashda muhim?

Docker — dasturni va uning barcha bog'liqliklarini (kutubxonalar, sozlamalar) alohida 'konteyner'ga joylashtirish vositasi.

Foyda: 'mening kompyuterimda ishlaydi' muammosi yo'qoladi — container hamma joyda bir xil ishlaydi.

`docker build -t loyiha .`

`docker run -p 5000:5000 loyiha`

Kalit: *Docker = dastur + muhit = portativ konteyner.*

2. CI/CD nima? Amaliy dasturlashda qanday qo'llaniladi?

CI (Continuous Integration) — kod har commit qilinganda avtomatik test o'tkaziladi.

CD (Continuous Deployment) — testlar o'tsa, avtomatik server'ga joylashtiriladi.

Misol: GitHub Actions bilan:

`git push` → test ishlaydi → o'tsa → Vercel'ga avtomatik deploy.

Kalit: *CI/CD = qo'lda ish yo'q, avtomatik test + chiqarish.*

3. ML loyihasida feature engineering nima va nima uchun muhim?

Feature engineering — xom ma'lumotdan model uchun foydali xususiyatlar (features) yaratish.

Misol: sana ustunidan yil, oy, hafta kuni, dam olish kuni ekanligi alohida ustun sifatida ajratiladi.

Yaxshi features — oddiy model bilan ham yaxshi natija berishi mumkin.

Kalit: *Feature engineering = to'g'ri savol berish = yaxshi model.*

4. Yil davomida o'rganilgan texnologiyalardan biri kelajak kasbingizga qanday ta'sir qiladi?

Shaxsiy javob: "[Texnologiya]ni o'rganib, [sohada] ishlashni xohlayapman, chunki..."

Kalit: *Mantiqiy fikr yuritish va maqsad belgilash muhim.*

YAKUNIY LOYIHA — 20 ball (baholash rubrikasi)

- 1. Loyiha muammosi aniq, dataset real va auditoriya belgilangan (5 ball) — Kalit: ML, ma'lumotlar tahlili yoki web+ML integratsiyasi bo'lsin.
- 2. Texnik qism: ishlaydigan ML model yoki data pipeline, baholash metrikalari ko'rsatilgan (8 ball) — Maksimal ball: to'liq ishlaydi, accuracy/F1 ko'rsatilgan.
- 3. Taqdimot: ilmiy konferensiya uslubida — muammo, usul, natija, xulosa (5 daqiqa) (5 ball)
- 4. Hamkorlik: agar guruhda bo'lsa, rol taqsimoti ko'rsatilsin (2 ball)

Ushbu hujjat «Kelajak texnologiyalari» to'garagi uchun tayyorlangan rasmiy o'qituvchi qo'llanmasi. Xo'jayev Muhammad Yusuf, Farg'ona viloyati, O'zbekiston tumani, Qumbosti MFY, 45-sonli maktab, 8-sinf. 2026-yil.